Technical Requirements Document

Time-Off Microservice

Field	Value
Document Version	1.1
Status	Complete
Author	Awais Ur Rehman
Date	April 2026
Stack	NestJS · SQLite · TypeScript
Reviewers	Engineering, Product

1. Executive Summary

ReadyOn's leave management module needs a backend service that handles the full lifecycle of employee time-off requests while keeping leave balances in sync with an external Human Capital Management system. The HCM is the authoritative source for leave balances, but it changes those balances independently of anything ReadyOn does. Anniversaries, year-start accruals, and manual adjustments in the HCM all happen without warning.

The service sits between ReadyOn's frontend and the HCM. It keeps a local shadow of HCM balances, runs defensive balance checks before touching the HCM, manages request state through an explicit state machine, and syncs with the HCM through three complementary paths to cover every scenario where the two systems can disagree.

The central tradeoff is accepting eventual consistency between the local shadow and the HCM in exchange for fast response times and resilience to HCM availability issues. That tradeoff is the load-bearing assumption in every section that follows.

2. Background

ReadyOn currently handles the employee-facing side of leave requests but relies on the HCM for balance data. The HCM exposes two interfaces: a real-time API for querying or writing individual balances, and a batch endpoint that delivers the full corpus of balances to ReadyOn. The HCM also pushes balance changes when external events occur, such as work anniversary bonuses or year-start accruals.

Keeping two systems in agreement is hard by default. A request submitted in ReadyOn must eventually be reflected in the HCM, and a balance change made in the HCM must eventually be visible in ReadyOn. When those updates happen concurrently, or when a network call fails partway through, the two systems end up disagreeing.

The harder problem is that the HCM does not guarantee it will reject invalid requests. The requirement explicitly states it "may not always" return errors for requests against insufficient balances. ReadyOn cannot treat the HCM as a gatekeeper. It has to enforce balance rules itself and treat any HCM error as a secondary check, not the primary one.

3. Goals and Non-Goals

In scope: REST API for submitting, reviewing, approving, rejecting, and cancelling time-off requests. Local shadow store for HCM balance data per employee per location. A reservation system to prevent concurrent over-deduction. A state machine for the full request lifecycle. Outbox-based async communication with the HCM to handle partial failures. Three-path sync architecture covering batch ingestion, webhook ingestion, and scheduled reconciliation. A mock HCM server for integration testing. Defensive local balance validation that operates independent of HCM error responses.

Out of scope: Authentication and user identity management. The service receives a validated employee or manager identity on each request via header, set by an upstream auth gateway. Frontend components. Push notifications to employees. Leave type configuration management. Multi-tenant support (single organization assumed throughout).

4. User Requirements

Employees need an accurate available balance at all times and immediate feedback on submission. The response to a submit request should either confirm the request was received or explain why it was blocked, without the employee waiting on an HCM round-trip.

Managers need to approve requests with confidence the balance data is current. Once they approve, the system should handle HCM confirmation without any further action from them. If confirmation fails, they get notified -- they should not be left looking at a request in a state they cannot interpret.

The system needs to stay consistent with the HCM even when the HCM changes balances outside of ReadyOn's control, and to handle partial failures without requiring manual intervention in the common case.

5. Proposed Solution

5.1 Architecture Overview

The service has six NestJS modules with clear ownership boundaries.

BalanceModule owns the local shadow of HCM balances. Each shadow record carries three counters: `total_days` (what the HCM has granted), `used_days` (confirmed deductions), and `reserved_days` (locally committed but not yet HCM-confirmed). The effective available balance shown to an employee is `total_days - used_days - reserved_days`. This module also owns the write endpoints that the sync paths call when HCM data arrives.

RequestModule owns the time-off request lifecycle. When a request is submitted, it atomically increments `reserved_days` on the balance row and creates the request record in a single transaction. Every state transition goes through an explicit guard before any write occurs. Cancellation and rejection release reserved days and write compensation events to the outbox if needed.

OutboxModule provides the outbox table and the processor that polls it. The processor runs on a configurable tick interval, picks up PENDING events, calls the HCM through the HcmClientModule, and handles success/failure/retry logic. Other modules write to the outbox within their own database transactions, so the intent to call the HCM is always committed atomically with the state change that triggers it.

SyncModule handles all inbound data from the HCM. Two endpoints receive HCM pushes (batch and real-time webhook). A scheduled job handles stale records by pulling from the HCM real-time API.

HcmClientModule is a thin HTTP wrapper around the HCM API. Retries with exponential backoff, timeout enforcement, and response normalization live here. No other module calls the HCM directly. Keeping it isolated makes it easy to swap the implementation in tests and easy to reason about what the system does when the HCM misbehaves.
EmployeeModule owns the employee entity. Thin CRUD, no business logic.

5.2 Request Lifecycle State Machine

```
┌──────────────┐
│  PENDING │   ← Initial state on submission
└──────┬───────┘
       Manager  │
       approves │
       ┌──────▼──────────────────────┐
       │ APPROVED_PENDING_HCM │  ← Balance reserved, HCM_DEDUCT queued
       └──────┬──────────────────────┘
   HCM confirms │          │ HCM fails (all retries exhausted)
       ┌──────▼──────┐  ┌──▼──────────┐
       │ APPROVED │  │ FAILED  │
       └─────────────┘  └──────┬──────────┘
                              │ Admin retry (outbox reset → HCM succeeds)
                          ┌──────▼──────┐
                          │ APPROVED │
                          └─────────────┘
```

Manager rejects (from PENDING) → REJECTED (reserved days released)

Employee/manager cancels (from PENDING or APPROVED_PENDING_HCM) → CANCELLED

If state was APPROVED_PENDING_HCM: HCM_REVERSE written to outbox

Terminal states: APPROVED, REJECTED, CANCELLED. These cannot transition further.

FAILED is a near-terminal state. It can only transition to APPROVED through the admin retry recovery path — an admin resets the FAILED outbox event to PENDING, the processor retries the HCM call, and on success the request moves FAILED → APPROVED.

Each transition has a guard. PENDING to APPROVED_PENDING_HCM requires `effective_balance >= days_requested`, checked atomically with the balance write. APPROVED_PENDING_HCM to APPROVED requires the corresponding outbox event to have completed. CANCELLED from APPROVED_PENDING_HCM writes a reversal event before releasing the reservation, because the deduction event in flight may still succeed — the reversal uses the `hcm_request_id` that the deduction eventually writes back to the request row.

The `version` column on `time_off_requests` enforces optimistic locking. Every state transition update includes `WHERE version = :currentVersion`. A zero row count means a concurrent modification won. The application retries or returns a 409.

5.3 Concurrency Control

The race condition that matters: two submissions for the same employee and location arrive within milliseconds of each other, both read the same available balance before either commits, and both pass a balance check that only one should pass.

SQLite in WAL mode allows concurrent readers and serialized writers. The balance deduction uses a conditional UPDATE rather than a read-then-write:

```
UPDATE leave_balances

SET reserved_days = reserved_days + :requested,

   version       = version + 1

WHERE employee_id = :eid

  AND location_id = :lid

  AND leave_type  = :type

  AND (total_days - used_days - reserved_days) >= :requested

  AND version     = :currentVersion
```

If this affects zero rows, either the balance is insufficient or a concurrent write already incremented the version. The application retries with a fresh balance read up to three times.

After three consecutive zero-row updates it returns a 409 with the current effective balance, so the client can decide what to show.

Additionally, a per-employee in-process serialization lock (promise-chaining keyed by `employeeId`) prevents concurrent async handlers from interleaving their transaction steps on `better-sqlite3`'s single synchronous connection. The optimistic locking in the database handles any remaining edge cases, but the in-process lock eliminates most version conflicts before they hit the database.

5.4 Sync Architecture

Three paths cover every scenario where the local shadow can disagree with the HCM.

Batch push (HCM → ReadyOn). The HCM delivers a full balance corpus to `POST /sync/batch`. The service upserts all records in a single transaction, reconciling incoming `total_days` values against local `used_days` and `reserved_days`. If HCM's balance for an employee is now lower than `used_days + reserved_days`, a discrepancy entry is written and the active requests for that employee are flagged. The sync log records how many records processed, how many discrepancies found, and whether the transaction committed.

Webhook push (HCM → ReadyOn). For individual real-time changes such as anniversary bonuses or year-start accruals, the HCM calls `POST /sync/webhook` with an employee ID, location ID, leave type, and updated balance values. The shadow record updates immediately. If the new `total_days` is lower than `used_days + reserved_days`, the same discrepancy and flagging logic runs as in the batch path.

Scheduled reconciliation. A cron job runs every 15 minutes. It queries the HCM real-time API for any shadow records where `last_synced_at` is older than 30 minutes or where there are active requests (PENDING or APPROVED_PENDING_HCM). This is the catch-all for missed webhooks and batch delays. If the reconciliation finds a discrepancy, it does not auto-resolve it; it logs and flags.

All three paths write to the same `leave_balances` table. The `last_synced_at` and `hcm_version` columns track what each record looked like the last time the HCM confirmed it.

5.5 Outbox Pattern

The outbox solves the dual-write problem. When the RequestModule needs to notify the HCM of a deduction or reversal, it does not call the HCM directly. Within the same database transaction as the state change, it inserts a record into `outbox_events`. If the transaction commits, the HCM will eventually be notified. If it rolls back, no HCM call will be made. There is no intermediate state where the request moved forward but the HCM never found out.

The OutboxProcessor runs on a configurable interval (default 5 seconds), picks up PENDING events where `next_retry_at` is null or in the past, marks each as PROCESSING (claim step to prevent double-processing), calls the HcmClientModule, and marks events DONE or increments their retry counter. Backoff schedule: attempt 1 immediately, attempt 2 after 30 seconds, attempt 3 after 5 minutes, attempt 4 (final) after 30 minutes. After the fourth failure the event status becomes FAILED and the associated request moves to FAILED status.

The completion and failure paths in the outbox processor are idempotent. If `completeDeductionFromOutbox` is called for a request that already reached APPROVED (e.g., idempotent replay after a crash), it returns early without error. If the request was CANCELLED while the deduction was in flight, the processor stores the `hcm_request_id` on the request (so the HCM_REVERSE event can reference it) without attempting a state transition. Similarly, `failRequestFromOutbox` is a no-op if the request is already in a terminal state.

The HCM client passes an idempotency key (derived from the outbox event ID: `outbox-{eventId}`) on every deduction call. This handles the specific failure where the HCM confirms but the local status update fails on the way back: the retry sends the same idempotency key, the HCM treats it as a duplicate, and the processor handles it as success.

5.6 HCM Reversal Race Condition

When a request is cancelled from APPROVED_PENDING_HCM, a HCM_REVERSE outbox event is created. But the original HCM_DEDUCT event may still be in flight or not yet processed. The reversal processor handles this by checking whether `hcm_request_id` has been set on the request row. If it hasn't (the deduction hasn't completed yet), the reversal throws an error and retries on the next tick. Once the deduction completes and writes `hcm_request_id`, the reversal picks it up and calls `DELETE /hcm/requests/:hcmRequestId` to undo the deduction.

This is a deliberate design choice: the deduction fires even for cancelled requests because the outbox event was already committed. The reversal follows behind and cleans up.

6. Data Model

`employees`

Column	Type	Notes
id	INTEGER PK	Internal identifier
hcm_employee_id	TEXT UNIQUE	HCM system identifier

Column	Type	Notes
name	TEXT	
location_id	TEXT	Primary location
created_at	DATETIME	

`leave_balances`

Column	Type	Notes
id	INTEGER PK	
employee_id	INTEGER FK	References employees
location_id	TEXT	
leave_type	TEXT	e.g., ANNUAL, SICK
total_days	DECIMAL	Latest value from HCM
used_days	DECIMAL	HCM-confirmed deductions
reserved_days	DECIMAL	Committed locally, not yet HCM-confirmed
last_synced_at	DATETIME	Timestamp of last HCM confirmation
hcm_version	TEXT	HCM etag/version for this record
version	INTEGER	Optimistic lock counter

Unique constraint on `(employee_id, location_id, leave_type)`.

Effective available = `total_days - used_days - reserved_days`.

`time_off_requests`

Column	Type	Notes
id	INTEGER PK	

Column	Type	Notes
employee_id	INTEGER FK	
location_id	TEXT	
leave_type	TEXT	
start_date	DATE	
end_date	DATE	
days_requested	DECIMAL	Calculated from date range
status	TEXT	State machine enum
manager_id	INTEGER FK	Assigned approver
rejection_reason	TEXT	Nullable
hcm_request_id	TEXT	Returned by HCM on confirmation
version	INTEGER	Optimistic lock counter
created_at	DATETIME	
updated_at	DATETIME	

outbox_events

Column	Type	Notes
id	INTEGER PK	
event_type	TEXT	HCM_DEDUCT or HCM_REVERSE
payload	TEXT	JSON
status	TEXT	PENDING, PROCESSING, DONE, FAILED
attempts	INTEGER	Default 0
next_retry_at	DATETIME	

Column	Type	Notes
request_id	INTEGER FK	Linked request
created_at	DATETIME	

`sync_log`

Column	Type	Notes
id	INTEGER PK	
sync_type	TEXT	BATCH, WEBHOOK, or SCHEDULED
triggered_by	TEXT	SYSTEM or admin user ID
status	TEXT	STARTED, DONE, or FAILED
records_processed	INTEGER	
discrepancies	INTEGER	Records flagged for review
error_detail	TEXT	Nullable
started_at	DATETIME	
completed_at	DATETIME	

7. API Specification

All responses are JSON. Error responses follow RFC 7807 (Problem Details), with a `type`, `title`, `status`, and `detail` field. Where a balance constraint is the reason for rejection, the error detail includes the current effective available balance.

7.1 Balance Endpoints

GET /balances/:employeeId Returns all leave types and locations for an employee. Each entry includes `total_days`, `used_days`, `reserved_days`, `effective_available`, and `last_synced_at`. Callable by the employee themselves or by a manager with scope over that employee's location.

GET /balances/:employeeId/:locationId Same as above, filtered to a single location.

7.2 Request Endpoints

POST /requests Submits a new time-off request. Body: `employeeId`, `locationId`, `leaveType`, `startDate`, `endDate`. The service calculates `days_requested` from the date range, runs the conditional balance update, and creates the request in PENDING state -- all in one transaction. If the balance is insufficient, returns 422 with the current effective available balance in the error detail. Returns 201 with the request ID and initial status on success.

GET /requests/:id Returns the full request record including current status, manager ID, rejection reason if applicable, and HCM request ID if the deduction has been confirmed.

GET /requests?employeeId=&status=&locationId= Paginated list with optional filters. Cursor-based pagination via `cursor` query parameter.

GET /requests/team?managerId=&status= Manager view of all requests from employees in their location scope. Returns the same request structure as `GET /requests/:id` plus employee name and effective available balance at the time of submission.

PATCH /requests/:id/approve Manager approves a PENDING request. Validates that the caller's manager scope covers the employee's location. Moves state to APPROVED_PENDING_HCM and writes a HCM_DEDUCT event to the outbox. Returns 200 immediately. The HCM call happens asynchronously.

PATCH /requests/:id/reject Manager rejects a PENDING request. Requires a `reason` field in the body. Moves state to REJECTED and releases `reserved_days`.

DELETE /requests/:id Cancels a request. If state is PENDING, releases reserved days with no HCM interaction. If state is APPROVED_PENDING_HCM, writes a HCM_REVERSE event to the outbox before releasing reserved days. Returns 204. Returns 409 if the request is already in a terminal state (APPROVED, REJECTED, CANCELLED, FAILED).

7.3 Sync Endpoints

POST /sync/batch Receives a full balance corpus from the HCM. Request body is an array of balance records. Runs as a single transaction. Returns 200 with `{ records_processed, discrepancies }`.

POST /sync/webhook Receives a single balance update from the HCM. Updates the shadow record and triggers discrepancy checking on active requests for that employee. Returns 204 on success.

POST /sync/trigger *(admin only)* Manually starts the scheduled reconciliation job. Useful after a known HCM outage or during testing.

GET /sync/status Returns the timestamp of the last successful batch sync, the last webhook received, outbox queue depth by status, and count of FAILED outbox events. Intended for monitoring dashboards and health checks.

7.4 Admin Endpoints

POST /outbox/:id/retry Manually retries a FAILED outbox event. Resets status to PENDING and `next_retry_at` to now. If the associated request is in FAILED state, the outbox processor will attempt the HCM call again and transition the request to APPROVED on success.

GET /outbox?status= Lists outbox events by status. Used for operational visibility.

8. Failure Modes and Mitigations

HCM is unavailable

Outbox events accumulate while the HCM is down and process in order when it recovers. Requests stay in APPROVED_PENDING_HCM, which is a meaningful state -- the manager approved, central confirmation is pending. After four failed attempts the event moves to FAILED, the reservation stays in place (the intent to deduct is still valid), and the request moves to FAILED status. An admin can retry the failed event through `POST /outbox/:id/retry`.

HCM confirms but the local status write fails

The outbox processor calls the HCM, receives success, then hits a database error updating the request status. On the next tick the processor retries. It sends the same idempotency key (`outbox-{eventId}`). The HCM returns either 200 (it processed it again and that is fine) or a response indicating it was already processed. Either way the processor treats it as success and retries the local write. The `completeDeductionFromOutbox` method is idempotent — if the request already reached APPROVED, it returns early without error.

Request cancelled while HCM deduction is in flight

When a request moves from APPROVED_PENDING_HCM to CANCELLED, the deduction outbox event may already be in PROCESSING. The deduction will succeed at the HCM. When the processor calls `completeDeductionFromOutbox`, it sees the request is CANCELLED and stores the `hcm_request_id` without transitioning state. The HCM_REVERSE event then picks up that `hcm_request_id` on its next retry and calls the HCM to reverse the deduction.

HCM balance drops below an existing reservation

The reconciliation path detects this: `total_days` from HCM is now lower than `used_days + reserved_days`. The service writes a discrepancy record and flags the active request records for that employee. It does not auto-cancel. The right action depends on business context -- whether the manager wants to override, cancel, or hold. The service surfaces the problem and waits.

Concurrent over-deduction attempt

The conditional UPDATE returns zero rows if the balance is insufficient or the version changed. The per-employee in-process lock prevents most contention, but when it does reach the database layer, the application retries with a fresh balance read up to three times. After three consecutive zero-row updates it returns 409 with the current effective balance, so the client can display "only X days available."

Database loss or corruption

SQLite with WAL mode is durable for a single-service workload. The batch sync endpoint is the recovery path after a restore: trigger it and the shadow catches up to HCM's current state. The sync_log and outbox tables provide enough information to determine which operations were in flight at the time of failure.

9. Alternatives Considered

Event Sourcing

Storing every state change as an immutable event rather than updating rows in place would produce a complete audit trail and make replay simple. The appeal is real. The problem is that event sourcing adds machinery -- event log compaction, projection rebuilding, schema evolution strategy -- that only pays for itself at scale or in a system with many independent consumers of the event stream. Neither applies here. The `sync_log` table and the `version` column on requests already cover auditability. Adding event sourcing would be over-engineering by the definition that matters: it would make the system harder to debug without making it more correct.

Always-synchronous HCM calls

The simpler path: when a manager approves, call the HCM directly in the same HTTP handler, wait, save the result, return. No outbox, no background processor.

The problem shows up the moment the HCM is slow. Manager approval takes as long as the HCM call. If the HCM times out at 8 seconds, every approval waits 8 seconds. If the HCM is

down for an hour, approvals are blocked for an hour. The outbox makes the manager's experience independent of HCM availability. That independence is worth the extra table and the background tick.

Skip local balance tracking, always query HCM

No shadow store. Every balance read hits the HCM real-time API. The sync complexity goes away entirely.

This breaks concurrency control. Without a local row to lock against, two concurrent submissions can both query the HCM, both see sufficient balance, and both get sent as deductions. The spec says the HCM may not catch this. Local balance tracking is the only mechanism available to enforce correct behavior under concurrent load. The sync complexity is not optional -- it is the cost of having a local lock target.

Redis and Bull for the outbox

A job queue backed by Redis would be cleaner for the async processor: built-in retry visibility, dead-letter queues, delay scheduling, and worker concurrency controls. There is nothing wrong with this choice.

The constraint is SQLite. Adding Redis means adding infrastructure not specified in the requirements. The outbox table in SQLite achieves the same delivery guarantees -- at-least-once, ordered per entity, retryable -- for the request volumes a time-off service sees. The tradeoff would be different if this service needed to process thousands of concurrent approvals.

GraphQL

The spec permits either REST or GraphQL. GraphQL would help a manager fetch team requests and current balances in one round-trip instead of two.

The operations in this service are imperative commands: submit, approve, cancel, reject. These map cleanly onto HTTP methods with predictable semantics. The one place GraphQL would save a round-trip is the manager's team view with embedded balances, which a single JOIN handles in SQL anyway. Adding GraphQL for that would mean schema definitions, resolvers, and a separate query language in the test suite, none of which improves correctness. REST covers the requirements without the overhead.

10. Security Considerations

An upstream auth gateway is assumed to validate identity and set `x-employee-id` and `x-role` headers before requests reach this service. The service trusts those headers and uses

them for access control: employees can read only their own data, managers can read and act on data within their location scope, and admin endpoints require the `admin` role.

The webhook endpoint at `POST /sync/webhook` should validate a shared secret in the `x-hcm-secret` header to prevent unauthorized balance updates. In the mock HCM and test environment, this is configurable via environment variable. In production, mutual TLS or OAuth would replace the shared secret without changing the handler logic.

No PII beyond employee names and IDs is stored in this service. Balance data is numerical. Leave dates are dates. There is nothing in the data model that requires encryption at rest beyond standard database file protection.

11. Testing Strategy

The requirement is clear on what this assessment values: test rigor over volume of code. The suite has three layers.

Unit Tests (95 tests)

Business logic tested in isolation with no database or network. The balance calculation (`total - used - reserved`), every state transition guard (both valid and invalid transitions), the optimistic lock retry logic, the outbox retry backoff schedule, and the reconciliation discrepancy detection. These run in milliseconds and catch regressions in pure logic without any environment setup.

Integration / E2E Tests (58 tests)

NestJS test module with in-memory SQLite plus the mock HCM server running as a real HTTP server on a local port. Each test suite uses `beforeEach` to reset database and mock HCM state so tests are fully isolated. Tests call the actual HTTP handlers via `supertest` and assert on both the response and the resulting database state.

Key scenarios covered:

Scenario	**What it proves**
Full happy path: submit → approve → HCM confirm	End-to-end flow commits correctly
Submit with insufficient balance	422 returned, no reservation written

Scenario	What it proves
Two concurrent submissions against same balance	Only one passes, second gets 409
HCM returns 500 on deduction	Request stays APPROVED_PENDING_HCM, outbox retries
HCM fails all four attempts	Request moves to FAILED, reserved days not released
Cancel before manager approval	Reserved days released, no outbox event
Cancel after manager approval (APPROVED_PENDING_HCM)	HCM_REVERSE written to outbox, deduction reversed
Batch sync reduces HCM balance below reservation	Discrepancy logged, active requests flagged
Webhook for anniversary bonus	total_days updated, effective_available recalculated
Scheduled reconciliation on stale record	HCM queried, shadow updated
Idempotent replay of outbox event	No double deduction on repeated HCM call
Admin retry of FAILED outbox event	Request recovers from FAILED → APPROVED

Mock HCM Server

A standalone NestJS application in `apps/mock-hcm`. It holds balance state in memory and exposes:

- `GET /hcm/balances/:employeeId/:locationId/:leaveType` -- current balance
- `POST /hcm/balances/batch` -- returns all in-memory balances
- `POST /hcm/requests` -- processes a deduction; respects `HCM_ERROR_RATE`; idempotent on repeated calls with the same `x-idempotency-key` header
- `DELETE /hcm/requests/:id` -- reversal
- `POST /hcm/simulate/anniversary/:employeeId` -- increases a balance to trigger the webhook test path
- `DELETE /hcm/state` -- reset all in-memory state (used in test setup/teardown)
- `GET /hcm/requests/:id` -- return a deduction record by ID (for verification)

In E2E tests, the mock HCM is managed as a shared singleton with reference counting. The first test suite to call `acquireMockHcm()` starts the server on port 3099; subsequent suites increment the reference count. When the last suite calls `releaseMockHcm()`, the server shuts down.

Coverage targets: 90% overall. 100% branch coverage on the balance update path and every state machine transition.

12. Implementation Phases

Three phases, all complete.

Phase 1 -- Foundation. Project scaffolding (NestJS monorepo with `apps/timeoff-service` and `apps/mock-hcm`), SQLite schema with TypeORM entity definitions, common enums, guards, filters, and the complete test file stubs written before any implementation code. Tests written against this spec, not reverse-engineered from the implementation.

Phase 2 -- Core Lifecycle. BalanceModule, RequestModule, state machine guards, per-employee serialization lock, and the concurrency-safe conditional balance UPDATE. Unit tests and integration tests for the happy path, balance validation, and concurrent submission scenarios.

Phase 3 -- Sync and Reliability. SyncModule (all three paths), OutboxProcessor with idempotent completion/failure handling, HcmClientModule with retry and idempotency key support. Docker multi-stage build and docker-compose for local development. All integration tests passing. Mock HCM stateful logic complete.

---

13. Open Questions

Manager-to-employee mapping. The spec does not define how managers are assigned to employees. This document assumes location-based scope (a manager oversees all employees sharing a location ID). If the actual model is team-based or hierarchical, the manager endpoints need adjustment.

Days calculation method. The spec does not define whether weekends and public holidays are excluded from `days_requested`. Calendar days are assumed. If business days are required, a holiday calendar input needs to be added to the submission payload or configured per location.

Leave type source. Leave types are treated here as unvalidated string identifiers passed in by the caller. If they need to be validated against a configured list, a leave type configuration endpoint and table need to be added.

Webhook authentication in production. The shared secret header approach works for testing. A production HCM integration (Workday, SAP) typically uses mutual TLS or an OAuth token grant. The webhook endpoint should be built to accept a pluggable auth strategy rather than a hardcoded header check.

Appendix A: Glossary

Term	Definition
HCM	Human Capital Management system (Workday, SAP, etc.). Authoritative source for employment and balance data.
Shadow store	Local SQLite copy of HCM balance data, kept current through the three sync paths.
Reserved days	Days committed against a balance after a request submits but before the HCM confirms the deduction.
Used days	Days confirmed deducted by the HCM.
Effective available	`total_days - used_days - reserved_days`. The balance employees and managers see.
Outbox	A database table used to deliver HCM operations reliably without dual-write failures.
Reconciliation	Comparing local shadow data against HCM data and writing discrepancy records where they disagree.

Appendix B: Environment Variables

Variable	Purpose	Default
DATABASE_PATH	SQLite file location	./data/timeoff.db
HCM_BASE_URL	HCM API base URL	http://localhost:3001
HCM_SECRET	Shared secret for webhook validation	(required)
OUTBOX_POLL_INTERVAL_MS	Outbox processor tick interval	5000
RECONCILIATION_CRON	Cron expression for scheduled reconciliation	`*/15 * * * *`
HCM_REQUEST_TIMEOUT_MS	Timeout for HCM HTTP calls	8000
HCM_MAX_RETRIES	Maximum outbox retry attempts	4
HCM_ERROR_RATE	Mock HCM only: random failure injection rate	0
PORT	HTTP server port	3000